DATABASE SYSTEMS

Comprehensive Study Notes

Topics 1 – 17 | Complete Reference

Covering: Fundamentals · SQL · Design · Transactions · NoSQL & More

Table of Contents

TOPIC 1

Introduction to Database Systems

1.1 What is a Database?

A **database** is an organised, shared collection of logically related data and a description of that data, designed to meet the information needs of an organisation. It stores data persistently so that it can be retrieved and manipulated efficiently.

1.2 Database Management System (DBMS)

A **DBMS** is software that enables users to define, create, maintain and control access to a database. It acts as an intermediary between the database and application programs or users.

Core functions of a DBMS:

- Data definition – create, alter, drop schema objects.
- Data manipulation – insert, update, delete, query data.
- Data security – access control and authorisation.
- Data integrity – enforce constraints and rules.
- Concurrency control – manage simultaneous access.
- Recovery management – restore after failures.

1.3 File-Based Systems vs DBMS

Aspect	File-Based System	DBMS
Data Redundancy	High – data duplicated across files	Controlled via normalisation
Data Inconsistency	Likely	Minimised
Data Sharing	Difficult	Easy & concurrent
Data Independence	None	Logical & physical independence
Query Capability	Hard-coded programs	Declarative query language
Security	Limited	Fine-grained access control

1.4 Advantages of DBMS

- Reduced data redundancy and inconsistency.
- Data sharing among multiple users and applications.
- Enforcement of data integrity constraints.
- Improved data security through access controls.
- Backup and recovery facilities.
- Support for multiple user interfaces.

1.5 Disadvantages of DBMS

- Higher initial cost (hardware, software, training).
- Increased complexity compared to flat files.
- Single point of failure if DBMS crashes (mitigated by replication).

- Performance overhead for simple, single-user applications.

1.6 Database Users & Roles

Role	Responsibilities
Database Administrator (DBA)	Schema definition, performance tuning, security, backup
Database Designer	Logical and physical schema design
Application Programmer	Writes programs that access the database
End User (naïve)	Uses applications without knowing SQL
End User (sophisticated)	Writes queries directly

TOPIC 2

Database System Architecture

2.1 ANSI/SPARC Three-Level Architecture

The ANSI/SPARC model divides a database system into three distinct levels of abstraction, providing **data independence**:

- **External Level (View Level):** Individual user or application views. Each user sees only the data relevant to them.
- **Conceptual Level (Logical Level):** The complete logical structure of the entire database – what data is stored and relationships among data.
- **Internal Level (Physical Level):** How data is physically stored (file organisation, indexes, access paths).

2.2 Data Independence

- **Logical Data Independence:** Ability to change the conceptual schema without changing the external schemas. e.g., adding a new attribute.
- **Physical Data Independence:** Ability to change the internal schema without changing the conceptual schema. e.g., changing file organisation.

2.3 Database Languages

- **DDL (Data Definition Language):** `CREATE, ALTER, DROP` – defines schema.
- **DML (Data Manipulation Language):** `SELECT, INSERT, UPDATE, DELETE` – manipulates data.
- **DCL (Data Control Language):** `GRANT, REVOKE` – access control.
- **TCL (Transaction Control Language):** `COMMIT, ROLLBACK, SAVEPOINT` – transaction management.

2.4 Centralised vs Client-Server Architecture

Architecture	Description	Use Case
Centralised	Single machine hosts DBMS & data	Small organisations
Client-Server (2-tier)	Client sends requests; server executes	LAN applications
Three-tier	Client, Application Server, DB Server	Web applications
N-tier	Multiple layers (microservices)	Enterprise / cloud systems

2.5 DBMS Component Architecture

Main components of a DBMS engine:

- **Query Processor:** Parser, semantic analyser, query optimiser, query executor.
- **Storage Manager:** Buffer manager, file manager, disk space manager.
- **Transaction Manager:** Concurrency control and recovery sub-systems.
- **Catalog / Data Dictionary:** Stores metadata about database objects.

TOPIC 3

Data Models

3.1 What is a Data Model?

A **data model** is a collection of concepts used to describe the structure of a database, the operations for manipulating data, and the constraints that the data must satisfy. It provides a framework for how data is organised, related, and accessed.

3.2 Types of Data Models

3.2.1 Hierarchical Model

Data is organised as a tree (parent-child relationships). A parent can have many children but each child has only one parent. Example: IBM IMS. Limitation: cannot represent many-to-many relationships naturally.

3.2.2 Network Model

An extension of the hierarchical model where records can have multiple parent and child records, forming a graph. Defined by the CODASYL standard. More flexible but complex to navigate.

3.2.3 Relational Model (Most Widely Used)

Proposed by E.F. Codd (1970). Data is represented as **relations (tables)** consisting of rows (tuples) and columns (attributes). Relationships between tables are expressed via common attribute values (foreign keys). Foundation for SQL.

3.2.4 Entity-Relationship (ER) Model

A **conceptual** data model that uses entities, attributes, and relationships to describe the real-world data structure. Used in the database design phase before mapping to the relational model.

- **Entity:** A real-world object (e.g., Student, Course).
- **Attribute:** Property of an entity (e.g., StudentID, Name).
- **Relationship:** Association between entities (e.g., Enrols).
- **Cardinality:** 1:1, 1:N, M:N

3.2.5 Object-Oriented Model

Data is represented as objects with attributes and methods, similar to OOP. Supports complex data types, inheritance, and encapsulation. Used in CAD, multimedia databases.

3.2.6 Object-Relational Model

Extends the relational model with OO features (user-defined types, inheritance, object identifiers). Supported by PostgreSQL, Oracle.

3.2.7 Semi-structured & Document Models

Used in NoSQL databases. Data need not conform to a fixed schema. Examples: JSON documents (MongoDB), XML.

3.3 ER Diagram Notation Summary

Symbol	Meaning
Rectangle	Entity

Ellipse	Attribute
Double ellipse	Multi-valued attribute
Dashed ellipse	Derived attribute
Diamond	Relationship
Double rectangle	Weak entity
Underlined attribute	Primary key

TOPIC 4

Relational Database Concepts

4.1 Terminology

Formal Term	Common Term	Description
Relation	Table	A set of tuples with the same attributes
Tuple	Row / Record	A single data record in a relation
Attribute	Column / Field	A named property of a relation
Domain	Data type / Range	Set of permissible values for an attribute
Degree	—	Number of attributes in a relation
Cardinality	—	Number of tuples in a relation

4.2 Keys

- **Super Key:** Any set of attributes that uniquely identifies a tuple.
- **Candidate Key:** Minimal super key (no redundant attributes).
- **Primary Key (PK):** Chosen candidate key to uniquely identify tuples; cannot be NULL.
- **Alternate Key:** Candidate keys not chosen as the primary key.
- **Foreign Key (FK):** An attribute (or set) in one relation that references the PK of another relation; enforces referential integrity.
- **Composite Key:** A key consisting of two or more attributes.

4.3 Integrity Constraints

- **Domain Constraint:** Attribute values must belong to the defined domain.
- **Entity Integrity:** Primary key attributes cannot be NULL.
- **Referential Integrity:** FK value must either match an existing PK or be NULL.
- **User-Defined Integrity:** Business rules (e.g., salary > 0).

4.4 Relational Schema Notation

A relation schema is written as: `RELATION_NAME (attr1, attr2, …)`

Example: `STUDENT (StudentID, Name, DOB, DeptID)` where `StudentID` is the PK and `DeptID` is a FK referencing `DEPARTMENT(DeptID)`.

4.5 Properties of Relations

- Each tuple is unique (no duplicate rows).
- Attribute values are atomic (First Normal Form).
- Column ordering is insignificant.
- Row ordering is insignificant.
- Each column has a unique name within the relation.

TOPIC 5

Relational Algebra & Calculus

5.1 Relational Algebra

Relational algebra is a **procedural** query language that uses a set of algebraic operations to manipulate relations. It forms the theoretical foundation of SQL.

5.2 Fundamental Operations

Operation	Symbol	Description	SQL Equivalent
Selection	σ (sigma)	Filter rows by a condition	WHERE clause
Projection	π (pi)	Select specific columns	SELECT (column list)
Union	∪	Combine tuples from two relations	UNION
Set Difference	–	Tuples in R1 but not in R2	EXCEPT / MINUS
Cartesian Product	×	All combinations of tuples	CROSS JOIN
Rename	ρ (rho)	Rename relation/attribute	AS alias

5.3 Derived Operations

Operation	Symbol	Description
Intersection	∩	Tuples common to both relations
Natural Join	■	Join on matching attribute names, remove duplicates
Theta Join	■θ	Join with an arbitrary condition (θ)
Equi-Join	—	Theta join with equality condition
Left Outer Join	■	All tuples from left + matching from right
Right Outer Join	■	All tuples from right + matching from left
Full Outer Join	■	All tuples from both with NULLs for non-matches
Division	÷	Find tuples in R1 that match all tuples in R2

5.4 Example Queries

Q1: Find all students in the CS department:

```
σ DeptName='CS' (STUDENT ■ DEPARTMENT)
```

Q2: Get student names and IDs only:

```
π StudentID, Name (STUDENT)
```

5.5 Relational Calculus

Relational calculus is a **declarative** query language that describes WHAT data to retrieve without specifying HOW. It comes in two forms:

- **Tuple Relational Calculus (TRC):** Variables range over tuples. Basis for SQL.

- **Domain Relational Calculus (DRC):** Variables range over attribute values. Basis for QBE (Query By Example).

Both are **relationally complete** – any query expressible in relational algebra can be expressed in relational calculus and vice versa.

TOPIC 6

Structured Query Language (SQL)

6.1 Overview

SQL (Structured Query Language) is the standard language for relational database management. It is **declarative** (you specify what you want, not how). SQL encompasses DDL, DML, DCL, and TCL.

6.2 DDL – Data Definition Language

CREATE TABLE:

```
CREATE TABLE Student ( StudentID INT PRIMARY KEY, Name VARCHAR(50) NOT NULL, DOB DATE, DeptID INT REFERENCES Department(DeptID) );
```

ALTER TABLE: `ALTER TABLE Student ADD COLUMN Email VARCHAR(100);`

DROP TABLE: `DROP TABLE Student;`

6.3 DML – Data Manipulation Language

SELECT (Basic):

```
SELECT Name, DOB FROM Student WHERE DeptID = 3 ORDER BY Name ASC;
```

INSERT:

```
INSERT INTO Student VALUES (101, 'Ali', '2002-05-10', 3);
```

UPDATE:

```
UPDATE Student SET DeptID = 4 WHERE StudentID = 101;
```

DELETE:

```
DELETE FROM Student WHERE StudentID = 101;
```

6.4 Aggregate Functions & GROUP BY

```
SELECT DeptID, COUNT(*) AS NumStudents, AVG(GPA) AS AvgGPA FROM Student GROUP BY DeptID HAVING COUNT(*) > 10;
```

Function	Description
COUNT()	Number of rows
SUM()	Total of numeric column
AVG()	Average value
MAX()	Maximum value
MIN()	Minimum value

6.5 JOINs

```
-- INNER JOIN SELECT S.Name, D.DeptName FROM Student S INNER JOIN Department D ON S.DeptID = D.DeptID; -- LEFT OUTER JOIN SELECT S.Name, D.DeptName FROM Student S LEFT JOIN Department D ON S.DeptID = D.DeptID;
```

6.6 Constraints

- `PRIMARY KEY` – unique, not null.
- `FOREIGN KEY` – referential integrity.
- `UNIQUE` – all values distinct.
- `NOT NULL` – no null values.
- `CHECK` – value must satisfy a condition.
- `DEFAULT` – default value when none supplied.

TOPIC 7

Advanced SQL

7.1 Subqueries

A **subquery** (nested query) is a query inside another SQL statement.

```
-- Single-row subquery SELECT Name FROM Student WHERE DeptID = (SELECT DeptID FROM Department WHERE DeptName = 'CS'); -- Multi-row subquery with IN SELECT Name FROM Student WHERE DeptID IN (SELECT DeptID FROM Department WHERE Location = 'Lahore'); -- Correlated subquery SELECT Name FROM Student S WHERE GPA > (SELECT AVG(GPA) FROM Student WHERE DeptID = S.DeptID);
```

7.2 Views

A **view** is a virtual table defined by a query. It simplifies complex queries, provides security, and offers logical data independence.

```
CREATE VIEW CS_Students AS SELECT StudentID, Name FROM Student WHERE DeptID = 3; SELECT * FROM CS_Students; -- use like a regular table
```

7.3 Stored Procedures & Functions

```
CREATE PROCEDURE GetStudentsByDept (IN p_DeptID INT) BEGIN SELECT * FROM Student WHERE DeptID = p_DeptID; END; CALL GetStudentsByDept(3);
```

7.4 Triggers

A **trigger** is a procedure that fires automatically before/after INSERT, UPDATE, or DELETE.

```
CREATE TRIGGER before_salary_update BEFORE UPDATE ON Employee FOR EACH ROW BEGIN IF NEW.Salary < OLD.Salary THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Salary cannot decrease'; END IF; END;
```

7.5 Window Functions

Window functions perform calculations across rows related to the current row without collapsing them.

```
SELECT Name, Salary, RANK() OVER (PARTITION BY DeptID ORDER BY Salary DESC) AS SalaryRank, SUM(Salary) OVER (PARTITION BY DeptID) AS DeptTotal FROM Employee;
```

7.6 Common Table Expressions (CTEs)

```
WITH TopStudents AS ( SELECT * FROM Student WHERE GPA >= 3.8 ) SELECT Name, GPA FROM TopStudents ORDER BY GPA DESC;
```

7.7 Indexes in SQL

Indexes speed up queries. `CREATE INDEX idx_name ON Student(Name);` — covered in depth in Topic 12.

7.8 CASE Expression

```
SELECT Name, CASE WHEN GPA >= 3.5 THEN 'Distinction' WHEN GPA >= 3.0 THEN 'Merit' ELSE 'Pass' END AS Grade FROM Student;
```

TOPIC 8

Database Design & Normalization

8.1 Design Phases

- **Requirement Analysis:** Understand user needs.
- **Conceptual Design:** ER diagram.
- **Logical Design:** Map ER to relational schema.
- **Normalisation:** Eliminate redundancy.
- **Physical Design:** Storage, indexes, partitioning.

8.2 Functional Dependency (FD)

An FD $X \rightarrow Y$ means that the value of X uniquely determines the value of Y. FDs are the basis of normalisation.

- **Trivial FD:** $X \rightarrow Y$ where $Y \subseteq X$.
- **Non-trivial FD:** $X \rightarrow Y$ where $Y \not\subset X$.
- **Armstrong's Axioms:** Reflexivity, Augmentation, Transitivity (used to derive all FDs from a given set).

8.3 Normal Forms

Normal Form	Requirement	Anomaly Eliminated
1NF	All attributes atomic; no repeating groups	Insertion of multi-valued data
2NF	1NF + no partial dependency (non-key attribute depends on full PK)	Partial dependency anomalies
3NF	2NF + no transitive dependency (non-key → non-key)	Transitive dependency anomalies
BCNF	For every FD X → Y, X must be a superkey	Anomalies not covered by 3NF
4NF	BCNF + no multi-valued dependencies	Multi-valued dependency anomalies
5NF	4NF + no join dependencies	Join decomposition anomalies

8.4 Example: Normalisation Steps

Unnormalised table: `ORDER(OrderID, CustName, {Items})` (repeating group)

1NF: `ORDER(OrderID, CustName, ItemID, ItemName, Qty)`

2NF: Remove partial deps → `ORDER(OrderID,CustName)` + `ORDER_ITEM(OrderID,ItemID,Qty)` + `ITEM(ItemID,ItemName)`

3NF: Verify no transitive deps remain.

8.5 ER-to-Relational Mapping Rules

- Each regular entity → relation with PK.
- Weak entity → relation with PK = partial key + owner PK (FK).
- 1:1 relationship → add FK to either side (prefer total participation side).

- 1:N relationship → add FK to the 'many' side.
- M:N relationship → create a new junction/bridge relation with FKs to both entities.
- Multi-valued attribute → separate relation with FK.

TOPIC 9

Transaction Management

9.1 What is a Transaction?

A **transaction** is a logical unit of work that consists of one or more database operations. It must be executed atomically – either all operations succeed or none take effect.

Example – Bank Transfer:

```
BEGIN TRANSACTION; UPDATE Account SET Balance = Balance - 500 WHERE AccID = 'A'; UPDATE Account SET Balance = Balance + 500 WHERE AccID = 'B'; COMMIT;
```

9.2 ACID Properties

Property	Meaning	Ensures
Atomicity	All or nothing	Incomplete transactions are rolled back
Consistency	DB moves from one valid state to another	Integrity constraints are maintained
Isolation	Concurrent transactions do not interfere	Intermediate states are invisible to others
Durability	Committed changes are permanent	Data survives system failures

9.3 Transaction States

- **Active:** Transaction is executing.
- **Partially Committed:** Last statement executed, awaiting commit.
- **Committed:** Transaction completed successfully; changes permanent.
- **Failed:** Normal execution cannot continue.
- **Aborted:** Transaction rolled back; DB restored to previous state.

9.4 Concurrency Problems Without Isolation

Problem	Description
Dirty Read	T2 reads uncommitted data written by T1; T1 then aborts
Non-Repeatable Read	T2 reads same row twice, gets different values (T1 updated in between)
Phantom Read	T2 re-executes a query; new rows appear because T1 inserted rows
Lost Update	T1 and T2 both read & update a row; T1's update is overwritten by T2

9.5 SQL Isolation Levels

Level	Dirty Read	Non-Repeatable Read	Phantom Read
READ UNCOMMITTED	Possible	Possible	Possible
READ COMMITTED	Prevented	Possible	Possible
REPEATABLE READ	Prevented	Prevented	Possible

SERIALIZABLE	Prevented	Prevented	Prevented

9.6 Schedules & Serializability

- **Serial Schedule:** Transactions execute one after another (no interleaving).
- **Serializable Schedule:** A concurrent schedule whose outcome is equivalent to some serial schedule.
- **Conflict Serializability:** A schedule is conflict-serializable if it can be transformed into a serial schedule by swapping non-conflicting operations.
- **Precedence Graph:** Used to test conflict serializability; if the graph has no cycle → serializable.

TOPIC 10

Concurrency Control

10.1 Goal

Concurrency control ensures that concurrent transactions execute correctly while maximising system throughput and minimising response time.

10.2 Lock-Based Protocols

- **Shared Lock (S):** Allows read; multiple transactions can hold S-locks simultaneously.
- **Exclusive Lock (X):** Allows read and write; no other lock can be granted concurrently.

Lock Compatibility Matrix:

	S (requested)	X (requested)
S (held)	Compatible ✓	Incompatible ✗
X (held)	Incompatible ✗	Incompatible ✗

10.3 Two-Phase Locking (2PL)

2PL guarantees conflict-serializability. Transactions must acquire all locks before releasing any.

- **Growing Phase:** Acquire locks; cannot release any.
- **Shrinking Phase:** Release locks; cannot acquire new ones.
- **Strict 2PL:** All exclusive locks held until commit/abort (prevents cascading rollbacks).
- **Rigorous 2PL:** All locks held until commit/abort.

10.4 Deadlock

A **deadlock** occurs when two or more transactions are waiting for each other to release locks.

Prevention strategies:

- **Wait-Die:** Older transactions wait; younger ones die (abort).
- **Wound-Wait:** Older transactions wound (abort) younger; younger wait.
- **No-Wait:** Abort immediately if a lock is not granted.

Detection: Maintain a wait-for graph; cycle = deadlock → abort one transaction (victim selection).

10.5 Timestamp-Based Concurrency Control

Each transaction T gets a unique timestamp TS(T). If conflict: older transaction wins.

- **Thomas's Write Rule:** Ignore outdated writes (optimisation).

10.6 Optimistic Concurrency Control (OCC)

Assume conflicts are rare. Transactions proceed without locks, then validate before commit. Three phases: **Read** → **Validate** → **Write**. Good for read-heavy workloads.

10.7 Multi-Version Concurrency Control (MVCC)

Each write creates a new version of the data item. Readers access a consistent snapshot without blocking writers. Used by PostgreSQL, Oracle, MySQL InnoDB.

TOPIC 11

Recovery Management

11.1 Types of Failures

- **Transaction Failure:** Logical error or system-detected error (deadlock); handled by rollback.
- **System Crash:** Power loss, OS crash, hardware fault; volatile memory lost.
- **Disk Failure:** Head crash or media failure; non-volatile storage lost.

11.2 Storage Hierarchy

- **Volatile Storage:** RAM – fast but lost on crash.
- **Non-Volatile Storage:** Disk/SSD – survives crashes.
- **Stable Storage:** Conceptual; information never lost (approximated via RAID + replication).

11.3 Log-Based Recovery

A **log** (write-ahead log, WAL) records every database modification before it is applied. Log records types:

- – transaction T started.
- – T updated X.
- – T committed.
- – T aborted.

Write-Ahead Logging (WAL) Rule: The log record must be written to stable storage before the data page is written to disk.

11.4 Undo and Redo

- **Undo:** Reverse changes of incomplete (failed) transactions using old values in the log.
- **Redo:** Re-apply changes of committed transactions that may not have been written to disk.

11.5 Checkpoints

Periodically write all modified buffers to disk and record a checkpoint in the log. On recovery, only transactions after the last checkpoint need to be processed, reducing recovery time.

11.6 ARIES Recovery Algorithm

The industry-standard recovery algorithm (used by DB2, SQL Server):

- **Analysis Pass:** Scan log from last checkpoint; determine dirty pages and active transactions at crash.
- **Redo Pass:** Redo all logged actions from the earliest dirty page LSN.
- **Undo Pass:** Undo incomplete transactions in reverse order.

11.7 Backup Strategies

Type	Description	Recovery Time
Full Backup	Complete copy of database	Fastest (single restore)
Differential	Changes since last full backup	Medium

Incremental	Changes since last backup (any)	Slowest (multiple restores)
Log Backup	Transaction log records	Used with full backup

TOPIC 12

Indexing & File Organization

12.1 File Organization

Organization	Description	Best For
Heap File	Records in insertion order	Bulk loads, full scans
Sorted File	Records sorted by search key	Range queries on sort key
Hashed File	Records in buckets by hash value	Equality searches
Clustered	Records physically ordered by index key	Range scans on index key

12.2 Indexing Concepts

An **index** is an auxiliary data structure that speeds up data retrieval at the cost of additional storage and update overhead.

- **Search Key:** Attribute(s) used to look up records.
- **Dense Index:** One index entry per data record.
- **Sparse Index:** One index entry per data block (only for sorted files).
- **Clustered Index:** Index order matches physical data order (at most one per table).
- **Non-Clustered (Secondary) Index:** Index order differs from physical order.

12.3 B+ Tree Index

The most widely used index structure. A balanced tree where all data pointers reside in leaf nodes, and leaf nodes are linked for range scans.

- Search: O(log N)
- Insert/Delete: O(log N) with automatic rebalancing.
- **Order n:** Each non-leaf node has between ■n/2■ and n pointers.

Leaf nodes store (search key, pointer to record) pairs and are linked by next-leaf pointers.

12.4 Hash Index

Uses a hash function h(k) to map search key k to a bucket.

- **Static Hashing:** Fixed number of buckets; overflow can occur.
- **Dynamic Hashing:** Buckets grow/shrink on demand.
- **Extendible Hashing:** Uses a global/local depth directory.
- **Linear Hashing:** Splits buckets linearly.

Limitation: Hash indexes only support equality queries, not range queries.

12.5 Bitmap Index

For each distinct value of a low-cardinality attribute, a bit vector with one bit per record. Efficient for data warehousing and analytical queries with multiple predicates (AND/OR via bitwise operations).

12.6 Index Selection Guidelines

- Index frequently queried columns (WHERE, JOIN, ORDER BY).
- Avoid over-indexing write-heavy tables.
- Use composite indexes for multi-column queries.
- Consider covering indexes (all needed columns in the index).

TOPIC 13

Query Processing & Optimization

13.1 Query Processing Steps

- **Parsing & Translation:** SQL → parse tree → relational algebra expression.
- **Optimisation:** Generate alternative execution plans; choose lowest-cost plan.
- **Evaluation:** Execute the chosen plan.

13.2 Cost Estimation

The query optimiser estimates cost using statistics stored in the system catalog:

- `n_r` – number of tuples in relation r.
- `b_r` – number of disk blocks for r.
- `V(A,r)` – number of distinct values of attribute A in r.
- `l_r` – size of a tuple.

13.3 Selection Algorithms

Algorithm	Cost (I/Os)	Condition
Linear scan	b_r	Always applicable
Primary B+ tree (equality)	h_i + 1	Key = value
Primary B+ tree (range)	h_i + range_blocks	A BETWEEN x AND y
Secondary B+ tree (equality)	h_i + n_matches	Non-key equality

13.4 Join Algorithms

Algorithm	Cost (simplified)	Best When
Nested-Loop Join	n_r × b_s	Small outer relation
Block Nested-Loop Join	b_r × b_s	Reduces I/Os vs NLJ
Indexed Nested-Loop	n_r × (I/O per lookup)	Index exists on inner
Sort-Merge Join	b_r + b_s + sort cost	Relations nearly sorted
Hash Join	3(b_r + b_s)	Large unsorted relations

13.5 Query Optimisation Techniques

- **Heuristic (Rule-Based) Optimisation:** Apply transformation rules:

1. Push selections down (apply early to reduce tuples).
2. Push projections down (reduce tuple width early).
3. Prefer natural joins over Cartesian products.
4. Reorder joins to process smallest intermediate results first.

- **Cost-Based Optimisation:** Enumerate plans using dynamic programming; pick minimum estimated cost.

- **Pipelined Evaluation:** Pass tuples between operators without materialising intermediate results (reduces I/O).
- **Materialised Evaluation:** Store intermediate results; needed when pipelining isn't possible (e.g., sort).

13.6 EXPLAIN / Query Plans

Use `EXPLAIN ANALYZE` in PostgreSQL or `EXPLAIN` in MySQL to inspect the chosen execution plan and actual run times.

TOPIC 14

Database Security

14.1 Security Threats

- Unauthorised reading (data theft / privacy breach).
- Unauthorised modification (data integrity violation).
- Denial-of-service (availability attack).
- SQL Injection attacks.

14.2 Access Control

Discretionary Access Control (DAC): Owner grants/revokes privileges.

```
GRANT SELECT, INSERT ON Student TO 'ahmed'@'localhost'; REVOKE INSERT ON Student FROM 'ahmed'@'localhost';
```

Mandatory Access Control (MAC): System-enforced labels (Top Secret > Secret > Confidential > Unclassified). Users can only access data at or below their clearance level.

Role-Based Access Control (RBAC): Privileges assigned to roles; users assigned to roles.

```
CREATE ROLE clerk; GRANT SELECT ON Orders TO clerk; GRANT clerk TO user_ali;
```

14.3 SQL Injection

SQL injection occurs when user input is concatenated into SQL queries without sanitisation.

Vulnerable: `"SELECT * FROM users WHERE name='" + input + "'"`

Exploit input: `' OR '1'='1`

Prevention: Use parameterised queries / prepared statements, input validation, principle of least privilege.

14.4 Encryption

- **Data at Rest:** Encrypt database files (AES-256). e.g., Transparent Data Encryption (TDE) in SQL Server/Oracle.
- **Data in Transit:** Use TLS/SSL for connections.
- **Column-Level Encryption:** Encrypt sensitive columns (PAN, SSN).
- **Hashing Passwords:** Store bcrypt/Argon2 hashes, never plaintext.

14.5 Auditing

Audit trails record who accessed or modified data and when. Essential for compliance (GDPR, HIPAA, PCI-DSS).

14.6 Views as Security Mechanism

Restrict access to sensitive columns by exposing only a view:

```
CREATE VIEW PublicEmployee AS SELECT EmpID, Name, Dept FROM Employee;
```

14.7 Statistical Database Security

In statistical databases (aggregate queries only), individual records should not be inferable. Techniques: adding noise (differential privacy), suppression, generalisation.

TOPIC 15

Distributed Databases

15.1 Definition

A **distributed database** is a collection of multiple, logically interrelated databases spread over a computer network. Users interact with it as if it were a single database.

15.2 Advantages & Challenges

Advantages	Challenges
Improved reliability/availability	Complex transaction management
Scalability (horizontal)	Concurrency control across nodes
Data locality (reduced latency)	Recovery from partial failures
Resource sharing	Security across distributed sites

15.3 Data Distribution

- **Replication:** Copies of data stored at multiple sites. Improves availability and read performance; increases update complexity.
- **Fragmentation:** Data partitioned across sites.

• **Horizontal:** Rows partitioned by predicate (e.g., region).

• **Vertical:** Columns partitioned across sites.

• **Mixed (Hybrid):** Combination of both.

15.4 Distributed Transactions

Require coordination across multiple sites. The **Two-Phase Commit (2PC)** protocol ensures atomicity:

- **Phase 1 – Prepare:** Coordinator asks all participants to vote YES/NO.
- **Phase 2 – Commit/Abort:** If all vote YES → commit; else → abort. Coordinator sends the decision to all.

Problem: Blocking protocol – if coordinator crashes during Phase 2, participants may wait indefinitely. **Three-Phase Commit (3PC)** addresses this.

15.5 CAP Theorem (Brewer's Theorem)

A distributed system can guarantee at most **two** of the following three properties simultaneously:

- **Consistency (C):** Every read receives the most recent write.
- **Availability (A):** Every request receives a response (not necessarily up-to-date).
- **Partition Tolerance (P):** System continues despite network partitions.

Since network partitions are unavoidable in practice, systems must choose between CP or AP.

15.6 Distributed Query Processing

Key cost factor: **data transfer cost** (network). Strategies include semi-joins to reduce data transferred and careful site selection for join operations.

TOPIC 16

NoSQL & Modern Databases

16.1 Motivation for NoSQL

- Massive scale (web-scale data) beyond single-machine capacity.
- Schema flexibility (semi-structured, evolving data).
- High availability and partition tolerance (CAP: AP systems).
- Diverse data types: documents, graphs, time-series, key-value pairs.

16.2 NoSQL Categories

Type	Data Model	Examples	Best Use Case
Key-Value Store	Key → arbitrary value	Redis, DynamoDB, Riak	Caching, sessions, leaderboards
Document Store	JSON/BSON documents	MongoDB, CouchDB, Firestore	Content mgmt, catalogs, user profiles
Column-Family	Column families	Cassandra, HBase, Bigtable	Time-series, write-heavy, IoT
Graph Database	Nodes + edges	Neo4j, Amazon Neptune, JanusGraph	Social networks, recommendations, fraud

16.3 BASE Properties (NoSQL)

Most NoSQL systems follow BASE instead of ACID:

- **Basically Available:** System guarantees availability (per CAP).
- **Soft State:** State may change over time without input (eventual consistency).
- **Eventually Consistent:** System will become consistent given no new updates.

16.4 MongoDB (Document Store) Quick Reference

```
// Insert db.students.insertOne({ name: "Ali", gpa: 3.8, dept: "CS" }) // Query db.students.find({ gpa: { $gte: 3.5 } }) // Update db.students.updateOne({ name: "Ali" }, { $set: { gpa: 3.9 } }) // Aggregation Pipeline db.students.aggregate([ { $match: { dept: "CS" } }, { $group: { _id: "$dept", avgGPA: { $avg: "$gpa" } } } ])
```

16.5 NewSQL Databases

NewSQL systems aim to provide the scalability of NoSQL while maintaining full ACID compliance and SQL support. Examples: Google Spanner, CockroachDB, NuoDB, VoltDB.

16.6 Time-Series Databases

Optimised for timestamped data streams (IoT sensors, financial ticks, monitoring metrics). Examples: InfluxDB, TimescaleDB, Prometheus.

16.7 RDBMS vs NoSQL Comparison

Feature	RDBMS	NoSQL
Schema	Fixed (rigid)	Flexible (schema-less)

Scalability	Vertical (scale-up)	Horizontal (scale-out)
ACID	Full support	Typically BASE
Query Language	SQL (standardised)	Varies by database
Joins	Native, efficient	Avoided or manual
Maturity	Decades of use	Relatively newer

TOPIC 17

Data Warehousing & Data Mining (Introductory)

17.1 Data Warehouse vs Operational DB

Aspect	Operational Database (OLTP)	Data Warehouse (OLAP)
Purpose	Day-to-day transactions	Decision support & analysis
Data	Current, detailed	Historical, summarised
Operations	Read/Write (INSERT, UPDATE)	Mostly Read (SELECT)
Design	Normalised (3NF)	Denormalised (Star/Snowflake)
Users	Clerks, operational staff	Analysts, managers, executives
Response Time	Milliseconds	Seconds to minutes
Volume	GB	TB to PB

17.2 Data Warehouse Architecture

- **ETL (Extract, Transform, Load):** Data extracted from source systems → cleaned/transformed → loaded into warehouse.
- **Staging Area:** Intermediate storage during ETL.
- **Data Mart:** Subject-oriented subset of a warehouse (e.g., Sales Mart).
- **Metadata Repository:** Describes structure, source, and transformation of warehouse data.

17.3 Dimensional Modelling

Fact Table: Central table containing quantitative measures (e.g., SalesAmount, Quantity). Contains FK references to dimension tables.

Dimension Table: Descriptive context (e.g., Time, Product, Customer, Store).

Star Schema: Fact table surrounded by denormalised dimension tables. Simple, fast queries.

Snowflake Schema: Dimension tables are normalised (sub-dimensions). Saves space but more complex queries.

17.4 OLAP Operations

- **Roll-Up:** Aggregate data (e.g., daily → monthly → yearly sales).
- **Drill-Down:** Navigate from summary to detail.
- **Slice:** Select one dimension value (e.g., Year = 2024).
- **Dice:** Select a sub-cube by multiple dimension values.
- **Pivot (Rotate):** Reorient the data cube for alternative presentations.

17.5 Introduction to Data Mining

Data mining is the process of discovering **patterns, correlations, anomalies, and insights** from large datasets using statistical and machine-learning techniques.

17.6 Data Mining Tasks

Task	Description	Example Algorithm
Classification	Assign items to predefined classes	Decision Tree, Naive Bayes, SVM
Clustering	Group similar items together	K-Means, DBSCAN, Hierarchical
Association Rules	Find frequent item co-occurrences	Apriori, FP-Growth
Regression	Predict continuous values	Linear Regression, Neural Networks
Anomaly Detection	Identify unusual patterns	Isolation Forest, LOF

17.7 KDD Process (Knowledge Discovery in Databases)

1. **Data Selection** → 2. **Preprocessing (Cleaning)** → 3. **Transformation** → 4. **Data Mining** → 5. **Interpretation / Evaluation**

17.8 Key Terms

- **Support:** Fraction of transactions containing an itemset.
- **Confidence:** P(Y | X) – probability Y is bought given X is bought.
- **Lift:** Confidence / P(Y) – measure of rule interestingness.
- **Big Data (5 V's):** Volume, Velocity, Variety, Veracity, Value.

Quick Reference Card

SQL Cheat Sheet

```
-- Basic Query SELECT col1, col2 FROM table WHERE cond GROUP BY col1 HAVING cond ORDER BY col1; -- Joins: INNER | LEFT | RIGHT | FULL OUTER | CROSS | NATURAL -- Set ops: UNION [ALL] | INTERSECT | EXCEPT -- Aggregate: COUNT | SUM | AVG | MAX | MIN -- Window: RANK() | DENSE_RANK() | ROW_NUMBER() | LAG() | LEAD() OVER(...) -- DDL: CREATE | ALTER | DROP | TRUNCATE -- DML: SELECT | INSERT | UPDATE | DELETE -- TCL: BEGIN | COMMIT | ROLLBACK | SAVEPOINT -- DCL: GRANT | REVOKE
```

Normal Forms Summary

UNF → 1NF (atomic) → 2NF (no partial deps) → 3NF (no transitive deps) → BCNF (every determinant is a superkey) → 4NF (no MVDs) → 5NF

ACID vs BASE

ACID: Atomicity, Consistency, Isolation, Durability — RDBMS transactions.

BASE: Basically Available, Soft state, Eventually consistent — NoSQL systems.

CAP Theorem

Choose 2 of 3: Consistency (C) | Availability (A) | Partition Tolerance (P). In practice, P is mandatory → choose CP or AP.

Index Types

B+ Tree: Range & equality | **Hash:** Equality only | **Bitmap:** Low-cardinality, OLAP | **Full-text:** Text search

NoSQL Types

Key-Value: Redis | **Document:** MongoDB | **Column-Family:** Cassandra | **Graph:** Neo4j